

What is a Kernel Memory Leak

A **memory leak** occurs when kernel code allocates memory but **never frees it**.

Example:

```
void *buf;  
  
buf = kmalloc(1024, GFP_KERNEL);  
  
/* forgot to call kfree(buf); */
```

If this happens repeatedly:

kmalloc → memory allocated
no kfree → memory lost forever

Eventually:

Out Of Memory
System slowdown
Kernel panic

2. Kernel Memory Leak Detection Tools

Linux provides several tools.

Tool	Purpose
Kmemleak	Kernel memory leak detector
KASAN	Kernel Address Sanitizer
SLUB Debug	Detect slab corruption/leaks
kmemtrace	Memory allocation tracing
kmalloc tracepoints	Perf / ftrace tracing

The **most common tool** is:

★ **Kmemleak**

3. How Kmemleak Works

Kmemleak works like a **garbage collector scanner**.

Steps internally:

- 1 Allocate memory
- 2 Track pointer
- 3 Scan kernel memory
- 4 Check references
- 5 If no reference → report leak

Conceptually:

Allocated Memory

|

v

Kernel pointer lost

|

v

kmemleak detects unreachable memory

4. Enable Kernel Memory Leak Detector

You must enable it in kernel configuration.

Step 1: Open Kernel Config

```
cd linux
```

```
make menuconfig
```

Go to:

Kernel hacking

->

Memory Debugging

->

Kernel memory leak detector

Enable:

```
CONFIG_DEBUG_KMEMLEAK=y
```

```
CONFIG_DEBUG_KMEMLEAK_EARLY_LOG_SIZE=400
```

Optional but recommended:

CONFIG_DEBUG_KMEMLEAK_TEST

Step 2: Rebuild Kernel

```
make -j8
make modules
make modules_install
make install
```

Reboot system.

Step 3: Verify

Check:

```
dmesg | grep kmemleak
```

Expected:

```
kmemleak: Kernel memory leak detector initialized
```

5. Kmemleak Interface

Kmemleak exposes an interface via **debugfs**.

Mount debugfs:

```
mount -t debugfs none /sys/kernel/debug
```

Check:

```
ls /sys/kernel/debug/kmemleak
```

Commands available:

Command	Meaning
---------	---------

scan	scan memory
------	----------------

clear	clear results
-------	---------------

dump	show leaks
------	------------

off	disable
-----	---------

on enable

6. Example Kernel Module With Memory Leak

Create a module.

leak_module.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/slab.h>

static void *leak_buf;

static int __init leak_init(void)
{
    printk("Leak module loaded\n");

    leak_buf = kmalloc(1024, GFP_KERNEL);

    return 0;
}

static void __exit leak_exit(void)
{
    printk("Leak module removed\n");

    /* Intentionally not freeing memory */

    /* kfree(leak_buf); */
}

module_init(leak_init);
module_exit(leak_exit);

MODULE_LICENSE("GPL");
```

7. Build Module

Makefile

obj-m += leak_module.o

all:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) modules

clean:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) clean

Compile:

make

8. Run the Leak

Load module:

sudo insmod leak_module.ko

Remove module:

sudo rmmod leak_module

Memory remains leaked.

9. Trigger Kmemleak Scan

Run:

echo scan > /sys/kernel/debug/kmemleak

Check leaks:

cat /sys/kernel/debug/kmemleak

Example output:

unreferenced object 0xffff888012345000 (size 1024):

comm "insmod", pid 1234

backtrace:

kmalloc

leak_init

do_one_initcall

Meaning:

Memory allocated
Pointer lost
Leak detected

10. Fix the Leak

Modify module:

```
static void __exit leak_exit(void)
{
    printk("Leak module removed\n");

    kfree(leak_buf);
}
```

Rebuild and run again.

Now:

```
echo scan > /sys/kernel/debug/kmemleak
cat /sys/kernel/debug/kmemleak
```

Output:

No leaks detected

11. Important Kmemleak Commands

Scan manually:

```
echo scan > /sys/kernel/debug/kmemleak
```

Clear old entries:

```
echo clear > /sys/kernel/debug/kmemleak
```

Disable:

```
echo off > /sys/kernel/debug/kmemleak
```

Enable:

```
echo on > /sys/kernel/debug/kmemleak
```

12. Using Kmemleak in Driver Development Workflow

Typical driver debugging workflow:

- 1 load driver
- 2 run workload
- 3 unload driver
- 4 scan for leaks

Example:

```
insmod driver.ko  
run test app  
rmmod driver
```

```
echo scan > kmemleak  
cat kmemleak
```

13. Typical Memory Leak Sources in Drivers

Very common mistakes:

1 Forgetting kfree

```
kmalloc()  
but no kfree()
```

2 Error Path Leak

```
buf = kmalloc()
```

```
if (error)  
    return -1
```

Fix:

```
if (error) {  
    kfree(buf);  
    return -1;  
}
```

3 Multiple allocations

```
buf1 = kmalloc()  
buf2 = kmalloc()
```

```
if (buf2 fails)  
    return error
```

Fix:

```
if (!buf2) {  
    kfree(buf1);  
    return error;  
}
```

14. Alternative Tool: KASAN

Another powerful tool:

Kernel Address Sanitizer

Enable:

CONFIG_KASAN

Detects:

memory leaks
buffer overflow
use-after-free

15. Real Kernel Debugging Workflow

Professional kernel debugging process:

Driver development

|

v

Enable debug kernel

|

v

Load module

|

v

Run workload

|

v

Check kmemleak

|

v

Fix leaks

16. Very Useful for Your Current Work

Since you are developing:

- GPIO drivers
- UART drivers
- DMA drivers
- ALSA drivers

on ****Raspberry Pi 4 Model B** using ****Linux Kernel 6.12**,

you should run **kmemleak scans after every driver unload**.

Example workflow:

```
insmod uart_dma.ko
```

```
run UART test
```

```
rmmod uart_dma.ko
```

```
echo scan > kmemleak
```

```
cat kmemleak
```